

MULTI TP5 Partage du bus dans les architectures multi-processeurs

Siyuan CHEN
Xiru ZHANG

March 2024

Table des matières

1	B. Architecture matérielle	2
1.1	Question B1	2
1.2	Question B2	2
2	C. Compilation de l'application logicielle	2
2.1	Question C1	2
2.2	Question C2	2
2.3	Question C3	2
3	D. Caractérisation de l'application logicielle	3
3.1	Question D1	3
3.2	Question D2	3
3.3	Question D3	3
4	E. Exécution sur architecture multi-processeurs	3
4.1	Question E2	3
4.2	Question E3	3
4.3	Question E4	4
5	F. Evaluation des temps d'accès au bus	4
5.1	Question F1	4
5.2	Question F2	4
5.3	Question F3	5
6	G. Modélisation du comportement du bus	5
6.1	Question G1	5
6.2	Question G2	6
6.3	Question G3	6

1 B. Architecture matérielle

1.1 Question B1

- Le nom du FrameBuffer. Ceci est une chaîne de caractères qui identifie le FrameBuffer dans le système.
- L'index comme esclave choisi par le processeur. Cet index indique la position du FrameBuffer dans la chaîne d'esclaves accessibles par le processeur.
- **Param3** : L'emplacement de la table des segments au périphérique. Ceci spécifie où les informations de segment sont stockées dans la mémoire du périphérique.
- **Param4** : Le nombre de cycles d'attente pour accéder par le bus. Ce nombre représente le délai d'accès au FrameBuffer via le bus en termes de cycles d'horloge.
- **Param5** : Le nombre de pixels par ligne. Cela définit la résolution horizontale du FrameBuffer.
- **Param6** : Le nombre de lignes dans un tampon. Cela indique la résolution verticale du FrameBuffer.
- **Param7** : Le format de pixel. Ce paramètre spécifie le format de couleur ou la profondeur de bit des pixels dans le FrameBuffer.

1.2 Question B2

La longueur du segment est de 64 k octets.

2 C. Compilation de l'application logicielle

2.1 Question C1

Le programme utilisateur n'a pas le droit d'accéder au noyau pour démarrer le frame buffer si le code se trouve dans l'espace système. Il faut donc un appel système pour demander au noyau de l'exécuter et de retourner ensuite au programme utilisateur. Sans l'appel système, le programme utilisateur ne pourrait jamais exécuter les fonctions du noyau.

2.2 Question C2

Parce que l'opération d'écriture dans un buffer est très coûteuse et qu'il n'est pas nécessaire d'écrire dans le buffer à moins qu'il y en ait besoin. C'est un mécanisme pour éviter les opérations d'écriture inutiles.

2.3 Question C3

- **Param1** : l'offset que l'on souhaite déplacer depuis la base du segment de buffer
- **Param2** : l'adresse de base du buffer

- **Param3** : la taille en octets qu'on veut écrire dans le buffer.

3 D. Caractérisation de l'application logicielle

3.1 Question D1

- Le nombre de cycles nécessaires pour afficher l'image avec un seul processeur est d'environ 4 555 779.
- Environ 3 822 693 instructions ont été exécutées pour afficher l'image.
- Le CPI est de 1,19.

3.2 Question D2

- Le pourcentage d'écritures par rapport à l'ensemble des instructions exécutées (taux d'écriture) est de 13
- Le pourcentage de lectures de données (taux de lecture) est de 25,1

3.3 Question D3

Le nombre de lectures est beaucoup plus élevé que le nombre d'écritures.

4 E. Exécution sur architecture multi-processeurs

4.1 Question E2

Les piles d'exécution sont strictement disjointes puisque les N tâches exécutées par N processeurs le sont de manière parallèle. Cette méthode permet de s'assurer que ces N processeurs ne modifient pas les données des autres tâches, et ainsi les tâches correspondent à nos attentes.

4.2 Question E3

En analysant le *Makefile* fourni, nous pouvons observer que :

1. Pour la compilation système, tous les fichiers, y compris le *driver*, le *handler*, et les fonctions communes, dépendent de `config.h`. Ce fichier a été modifié pour ajuster le nombre de processeurs.
2. Pour la compilation de l'application, la fonction principale qui se charge de la séparation du traitement d'une ligne de pixels dépend également du nombre de processeurs. Cette dépendance permet de répartir l'exécution des tâches en parallèle sur toutes les lignes.

4.3 Question E4

Selon la loi d'Amdahl, il existe un temps minimal pendant lequel la tâche ne peut pas s'exécuter en parallèle, qui ne peut pas être amélioré en ajoutant des processeurs.

TABLE 1 – Speedup table

	1	2	4	6	8
cycles	4555779	2290722	1189301	1023395	1036166
Speed up	1	1,98	3,83	4,45	4,39

5 F. Evaluation des temps d'accès au bus

5.1 Question F1

Au moment précis, l'exécution de programme se termine sans cycles non nécessaires qui augmente le coût des MISS et d'autre calcule qui utilise le nombre total du cycle.

5.2 Question F2

TABLE 2 – Sample three-line table

NPROCS	1	2	4	6	8
IMISS	15,85	17,8	31,6	42,8	62,3
COST					
DMISS	16,44	18,2	56,24	77,7	112,1
COST					
WRITE	0	0	0,256	2,92	6,50
COST					
ACCESS.TIME	Proc0 : 1				
		Proc0 : 1,11	Proc0 : 2,80	Proc0 : 8,48	Proc0 : 12,59
		Proc1 : 1,22	Proc1 : 2,89	Proc1 : 8,45	Proc1 : 12,60
			Proc2 : 2,75	Proc2 : 8,47	Proc2 : 12,70
			Proc3 : 2,77	Proc3 : 8,49	Proc3 : 12,69
				Proc4 : 8,60	Proc4 : 12,69
				Proc5 : 8,60	Proc5 : 12,70
					Proc6 : 12,70
					Proc7 : 12,70
CPI	1,19	1,19	1,24	1,55	2,14

5.3 Question F3

Comparé au temps d'accès processeur du coût de MISS, le coût d'écriture est très bas. Pour le même traitement d'image, le CPI a doublé. Cela est dû à l'augmentation du coût de MISS qui augmente le nombre total de cycles.

6 G. Modélisation du comportement du bus

6.1 Question G1

Temps d'occupation pour chaque type de transaction :

- **IMISS** 6 cycles :
 - 1er cycle : Initialisation sur le bus.
 - 2e cycle : Obtenir la requête sur le bus.
 - 3e cycle : Obtenir l'adresse sur le bus.
 - 4e cycle : Attendre de donnée sur l'adresse correspondant sur le bus.
 - 5e cycle : Obtenir de donnée sur le bus.
 - 6e cycle : Cycle mort
- **DMISS** 6 cycles :
 - 1er cycle : Initialiser sur le bus.
 - 2e cycle : Obtenir la requête sur le bus.
 - 3e cycle : Obtenir l'adresse sur le bus.
 - 4e cycle : Attendre de donnée sur l'adresse correspondant sur le bus.
 - 5e cycle : Obtenir de donnée sur le bus.
 - 6e cycle : Cycle mort
- **UNC** 5 cycle :
 - 1er cycle : Initialisation sur le bus.
 - 2e cycle : Obtenir la requête sur le bus.
 - 3e cycle : Obtenir l'adresse sur le bus.
 - 4e cycle : Obtenir la donnée sur le bus.
 - 5e cycle : Cycle mort.
- **WRITE**
 - 1er cycle : Initialisation sur le bus.
 - 2e cycle : Obtenir la requête sur le bus.
 - 3e cycle : Obtenir l'adresse sur le bus.
 - 4e cycle : Obtenir la donnée sur le bus.
 - 5e cycle : Cycle mort.

6.2 Question G2

	Temp_occupation	Fréquence
IMISS	6	$6e - 5$
DMISS	6	$1,7e - 4$
UNC	5	$3,15e - 3$
WRITE	5	$0,06 * 5 = 0,3$

6.3 Question G3

Le nombre de processeur qui pourrait saturer le bus est :

$$\text{Nbr} = 1 / \sum_{i=1}^4 \text{fréquence de transaction}$$

$$= 1 / (\sim 0,3) = 3$$